

# Ansible Variables and Host Management - 1

Mastering data collection, variable management, and security in Ansible automation



# Understanding Ansible Facts

Facts are essential pieces of information about the systems that Ansible manages. They serve as the foundation for intelligent automation decisions and dynamic playbook behaviour.

When Ansible connects to a host, it automatically collects comprehensive system details including IP addresses, operating system versions, hardware specifications, network configurations, and much more. This automatic discovery enables you to write playbooks that adapt to different environments without hardcoding system-specific values.

## Gathering Facts Manually

```
ansible all -i inventory -m ansible.builtin.setup
```

This command displays all facts for hosts in your inventory, providing valuable insight into your infrastructure.



### Network Details

IP addresses, interfaces, DNS



### Hardware Info

CPU, memory, architecture



### OS Details

Distribution, version, kernel

# Accessing Fact Variables

All collected facts are stored within a special variable called `ansible_facts`. Understanding how to access these facts is crucial for building dynamic and responsive playbooks.

## Bracket Notation

The recommended approach for accessing facts:

{

```
ansible_facts['property_name']
```

This method is safer as it avoids conflicts with Python dictionary methods and handles keys containing special characters like periods.

## Dot Notation

An alternative syntax for simpler property names:

⓪

```
ansible_facts.property_name
```

Whilst more concise, this method can cause errors with certain key names, so bracket notation is generally preferred.

# Controlling Fact Gathering

## Default Behaviour

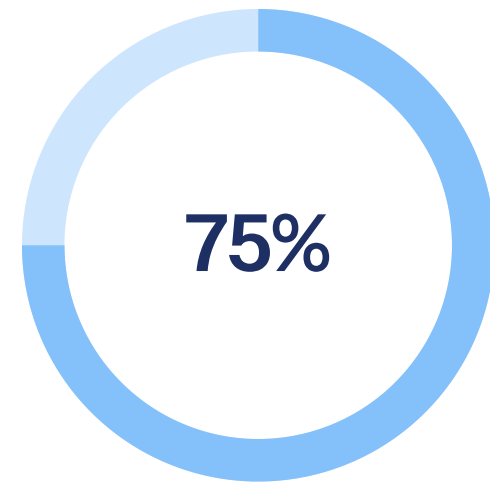
By default, Ansible sets `gather_facts: true`, automatically collecting system information at the start of every playbook run. This is convenient but adds execution time, particularly when managing numerous hosts.

## Disabling for Performance

Set `gather_facts: false` at the beginning of your playbook to skip automatic fact collection. This significantly speeds up execution when facts aren't needed.

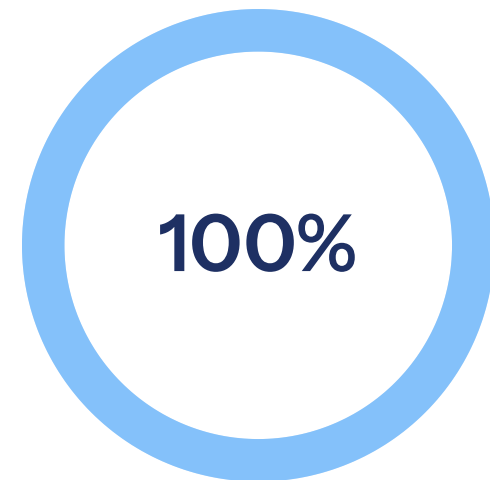
## Selective Gathering

If you disable automatic gathering but later need facts, you can use the `setup` module in a specific task to collect them on demand, giving you fine-grained control over when fact gathering occurs.



### Faster Execution

Potential speed improvement with fact gathering disabled



### Full Control

Decide exactly when facts are needed

❏ **Pro Tip:** For playbooks that only perform simple tasks like copying files or restarting services, disabling fact gathering can dramatically reduce execution time across large inventories.

# Simple Variables in Ansible

Variables are fundamental building blocks in Ansible, allowing you to store and reuse values throughout your playbooks. Simple variables store single values that can be referenced multiple times, promoting consistency and maintainability.

## String Variables

Store text values such as server names, paths, or configuration parameters

## Numeric Variables

Hold integers or floating-point numbers for ports, timeouts, or thresholds

## Boolean Variables

Ansible accepts multiple boolean formats: `true/false`, `1/0`, `yes/no`, `True/False`, `t/f`, `y/n`, or `on/off`



# Complex Variable Types

## List Variables

List variables combine a variable name with multiple ordered values, perfect for storing collections of related items such as package names, user accounts, or server addresses.

Lists can be defined using itemised YAML format or square bracket notation:

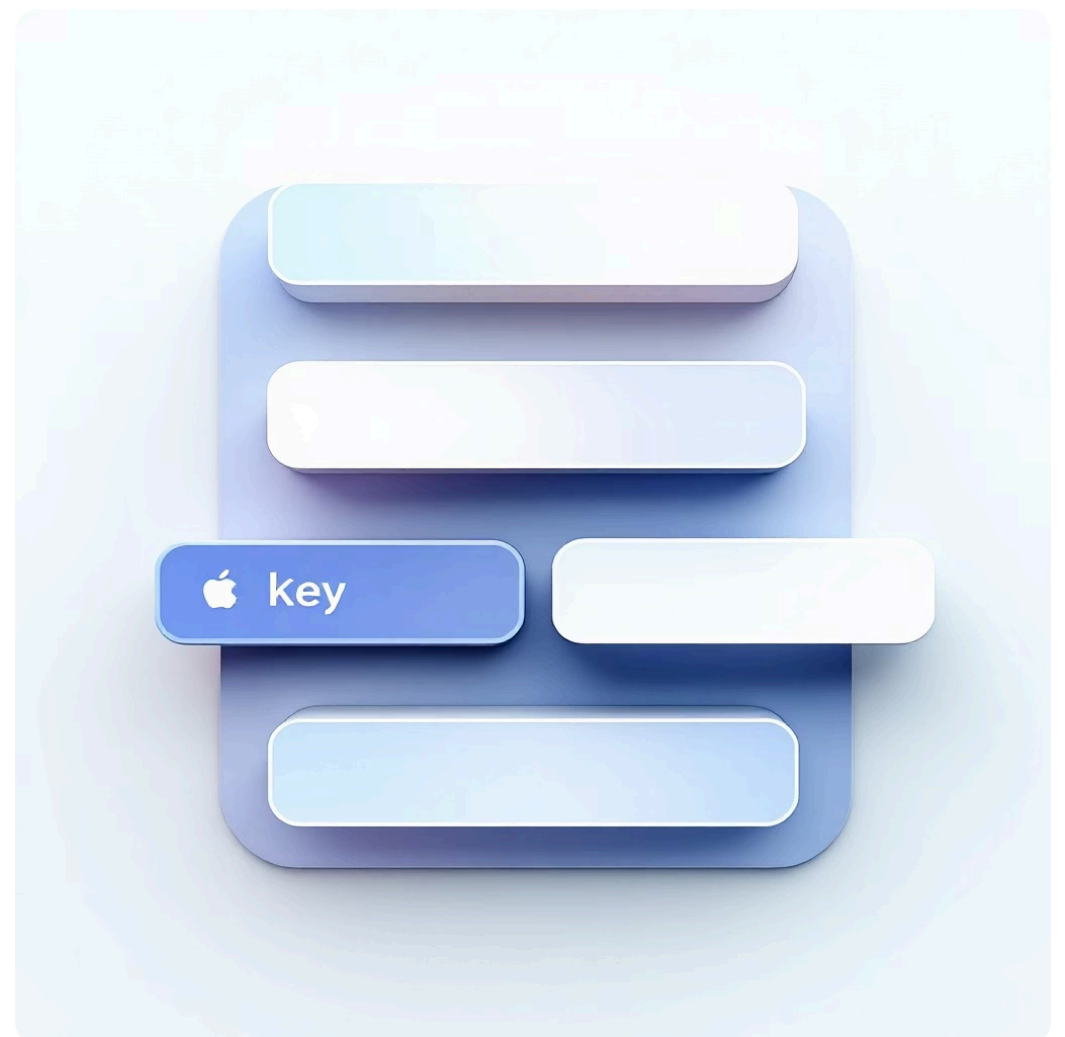
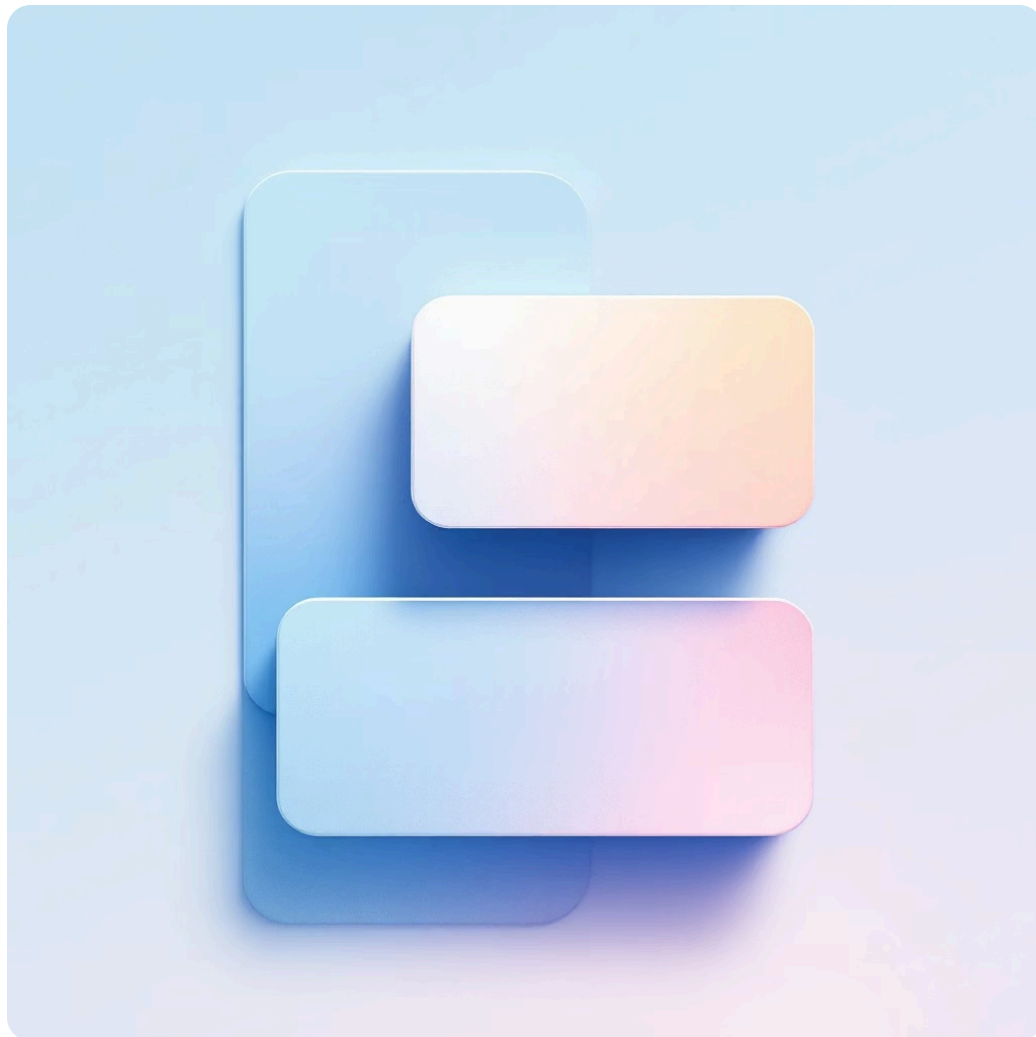
```
packages:  
- nginx  
- postgresql  
- redis  
  
# Or: packages: [nginx, postgresql, redis]
```

## Dictionary Variables

Dictionaries store data as key-value pairs, similar to maps in other programming languages. They're ideal for structured configuration data.

Access values using dot notation (`dict.key`) or bracket notation (`dict['key']`). Bracket notation is recommended to avoid conflicts with Python methods and handle keys with special characters.

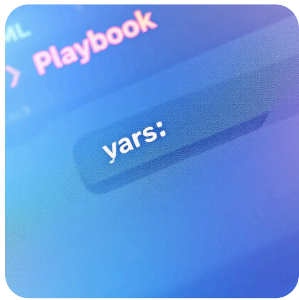
```
user_config:  
  name: admin  
  port: 8080  
  enabled: true  
  
# Access: user_config['name']
```





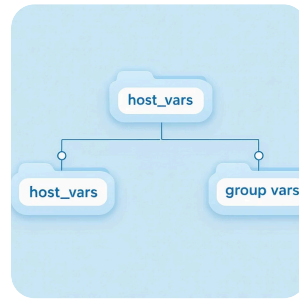
# Setting Variable Values

Ansible offers multiple locations for defining variables, each serving different use cases and organizational needs. Understanding where to place variables helps maintain clean, manageable automation code.



## Playbook Files

Define variables directly within playbooks using the `vars` section for task-specific values



## host\_vars and group\_vars

Organise variables in dedicated directories for specific hosts or groups



## Inventory

Set variables directly in inventory files alongside host definitions



## Runtime (Extra Vars)

Pass variables at execution time using `--extra-vars` for sensitive or dynamic values

📌 For sensitive values, you can hash the entered value to protect it from exposure whilst maintaining usability in your playbooks.

# Variable Precedence Hierarchy

When the same variable is defined in multiple locations, Ansible applies variable precedence rules to determine which value takes effect. Understanding this hierarchy is essential for predictable playbook behaviour.



The complete precedence order includes 22 levels, from role defaults at the bottom to extra vars at the top. Variables defined at higher levels always override those at lower levels, allowing you to set sensible defaults whilst maintaining flexibility for specific scenarios.

❏ **Remember:** Extra vars passed via `--extra-vars` always win, making them perfect for overriding any value at runtime.





# Protecting Sensitive Data with Ansible Vault

Ansible Vault provides robust encryption capabilities to protect sensitive data such as passwords, API keys, and certificates from exposure in playbooks, version control, or logs. This security feature is essential for maintaining proper security hygiene in your automation workflows.



## Create Encrypted Files

Use `ansible-vault create` to generate a new encrypted file for storing secrets



## Edit Securely

Modify encrypted content directly with `ansible-vault edit` without decrypting to disk



## Decrypt When Needed

Temporarily decrypt files using `ansible-vault decrypt` for inspection or migration

# Running Playbooks with Encrypted Data

When executing playbooks that contain Ansible Vault encrypted data, you must provide the vault password to decrypt the sensitive information at runtime.

## Interactive Password Prompt

Use the `--ask-vault-pass` option to prompt for the password during execution:

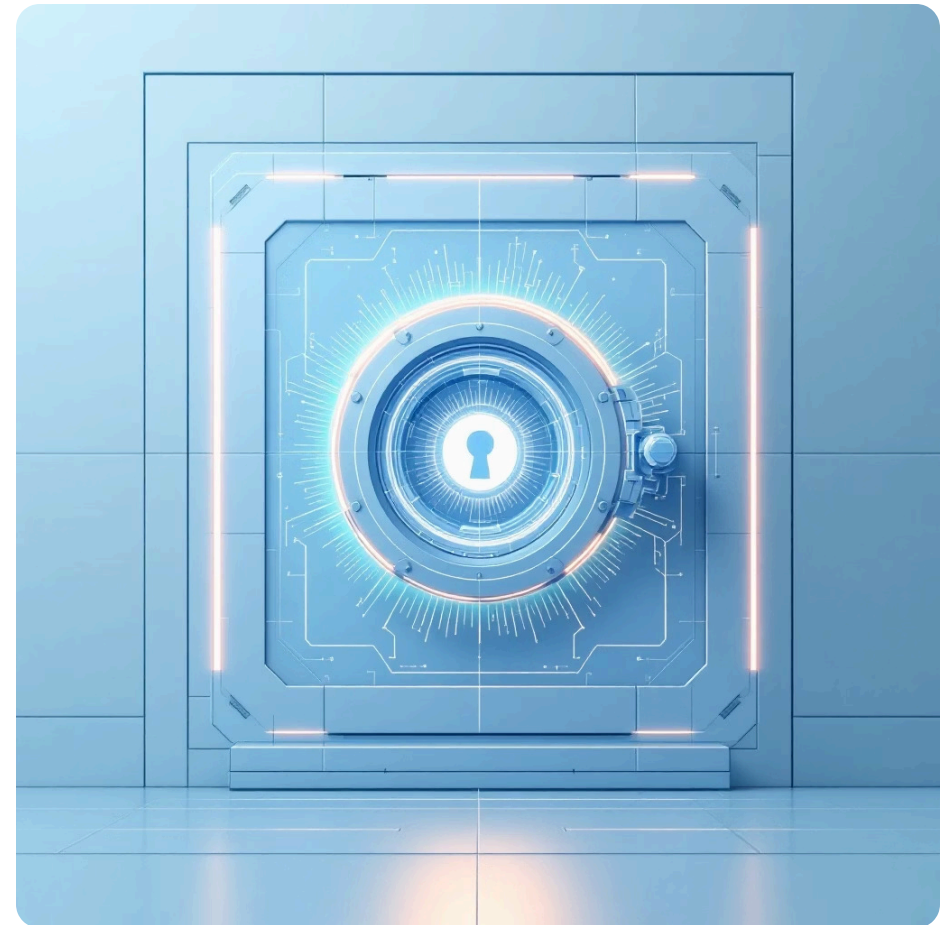
```
ansible-playbook playbook.yml --ask-vault-pass
```

## Password File Method

Alternatively, store the password in a file and reference it with `--vault-password-file` for automation scenarios where interactive prompts aren't feasible.

## Multiple Vault IDs

Ansible supports multiple vault passwords using vault IDs, allowing you to segregate secrets by environment or team whilst maintaining separate encryption keys for each.



### Best Practice

Never commit vault passwords to version control. Use environment variables or secure credential management systems.

### Audit Trail

Encrypted files can be safely committed to Git, providing version history whilst protecting sensitive values.